Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

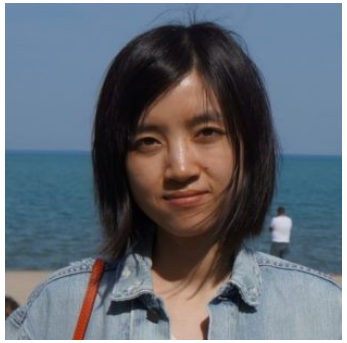


Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)
Cloud native and AI infrastructure

nunchux.ai · CNCF TOC



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

Different paths. Shared infrastructure.

01 Unified scheduling

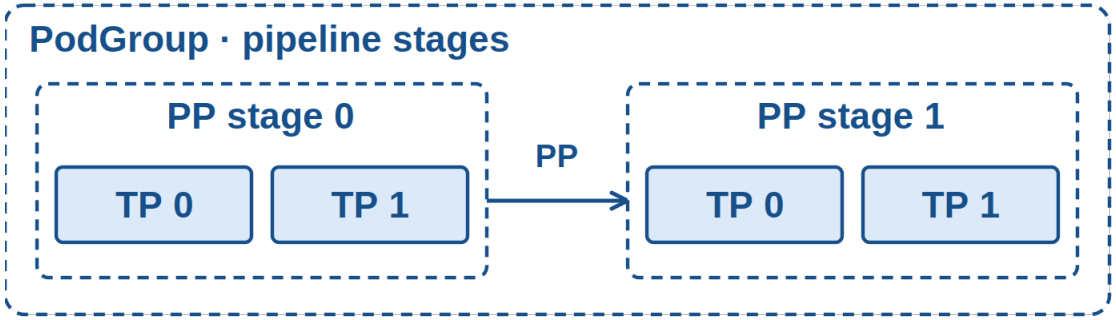
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

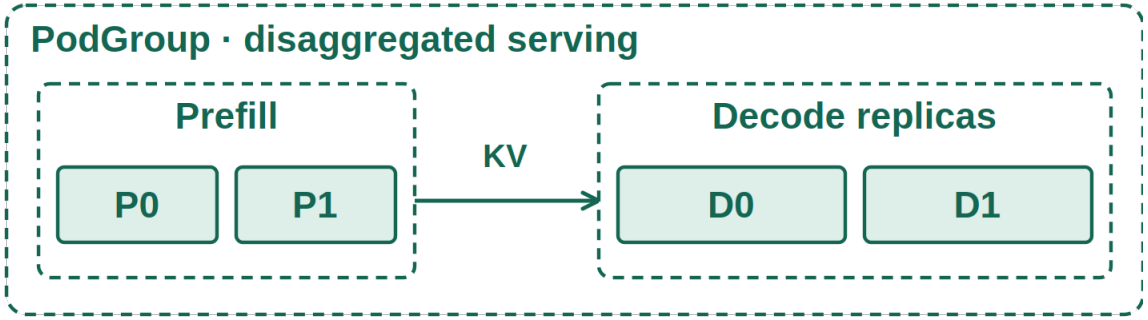
Different Workloads, Shared Infrastructure

Training



TP / PP gangs + network locality

Inference



Role gangs + P/D locality

Agents

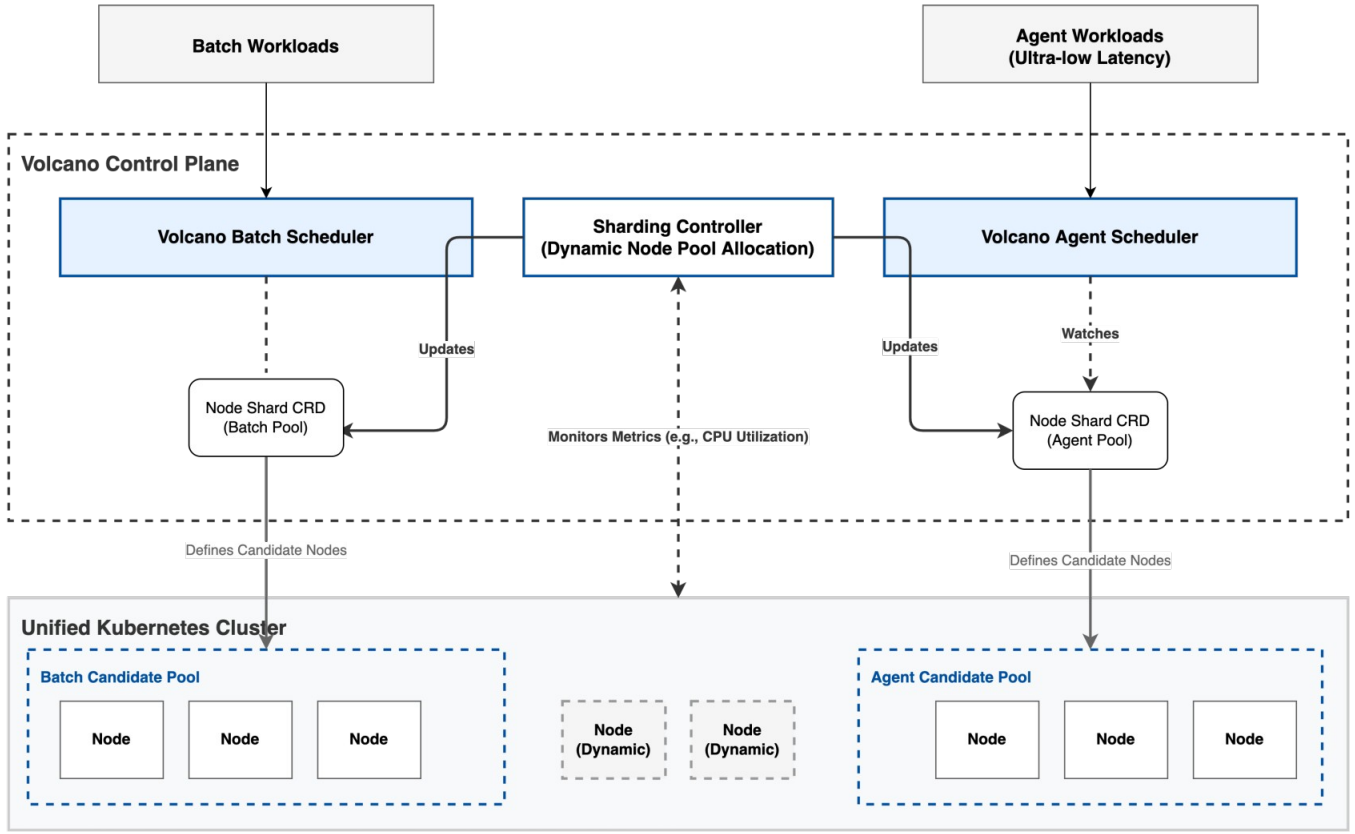
Bursty arrivals



Low startup latency



A Coordinated Scheduling Framework



Scheduling paths

Batch throughput / Pod latency

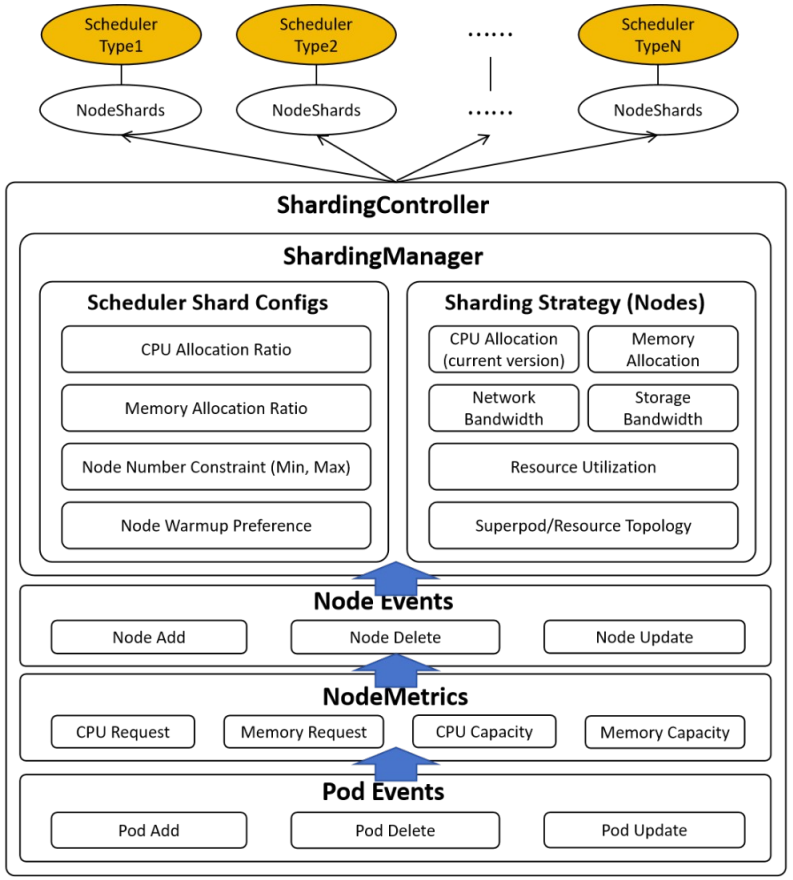
Worker coordination

Parallel workers + Binder checks

NodeShard

Coordinated node access

NodeShard: Desired Nodes and Current Use



NodeShard · spec / status excerpt

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Coordinated handoff · no Pod migration

02

Multi-level gang & network topology

From available nodes to runnable workloads.

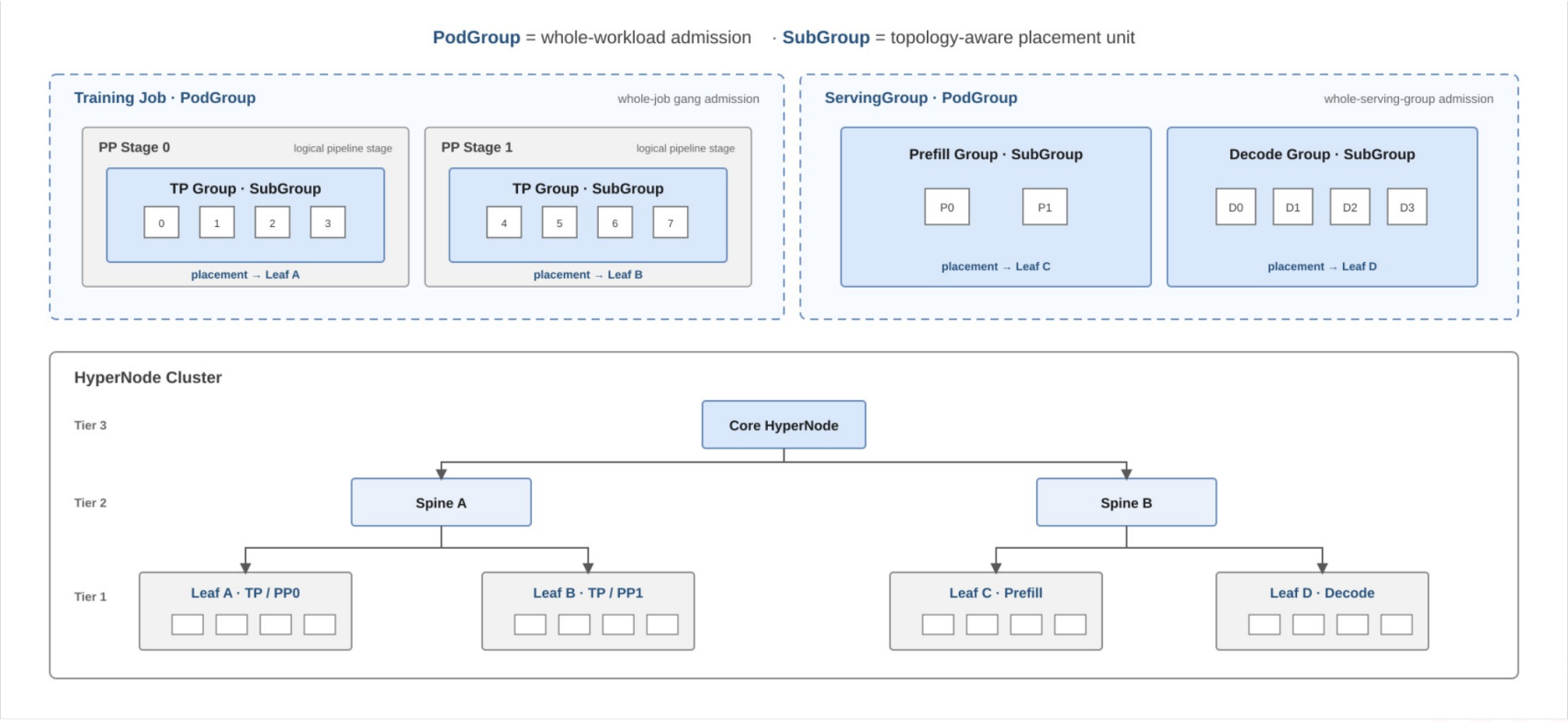
01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

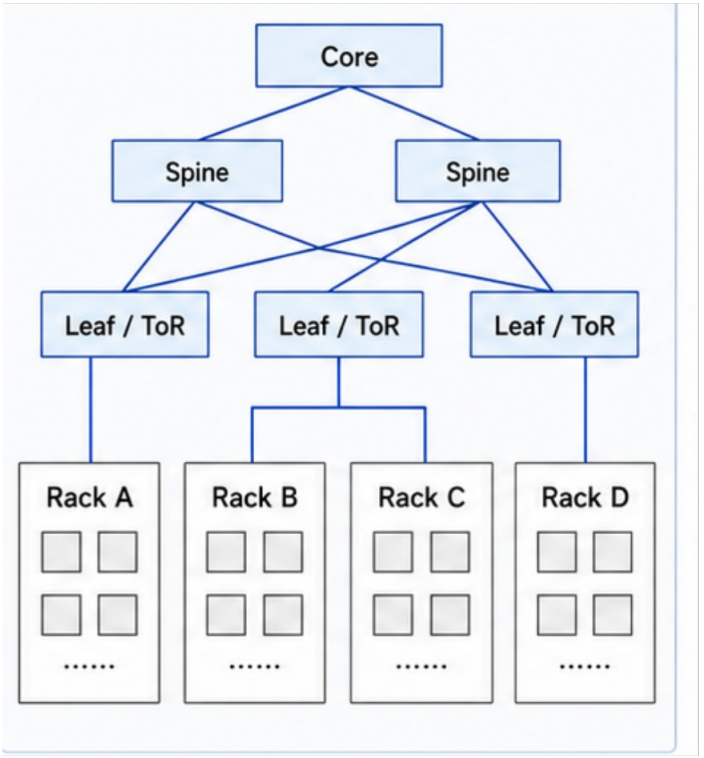
04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure



Replica identity → complete SubGroups

HyperNode: From Physical Fabric to Scheduling Domains



Parent · HyperNode.spec

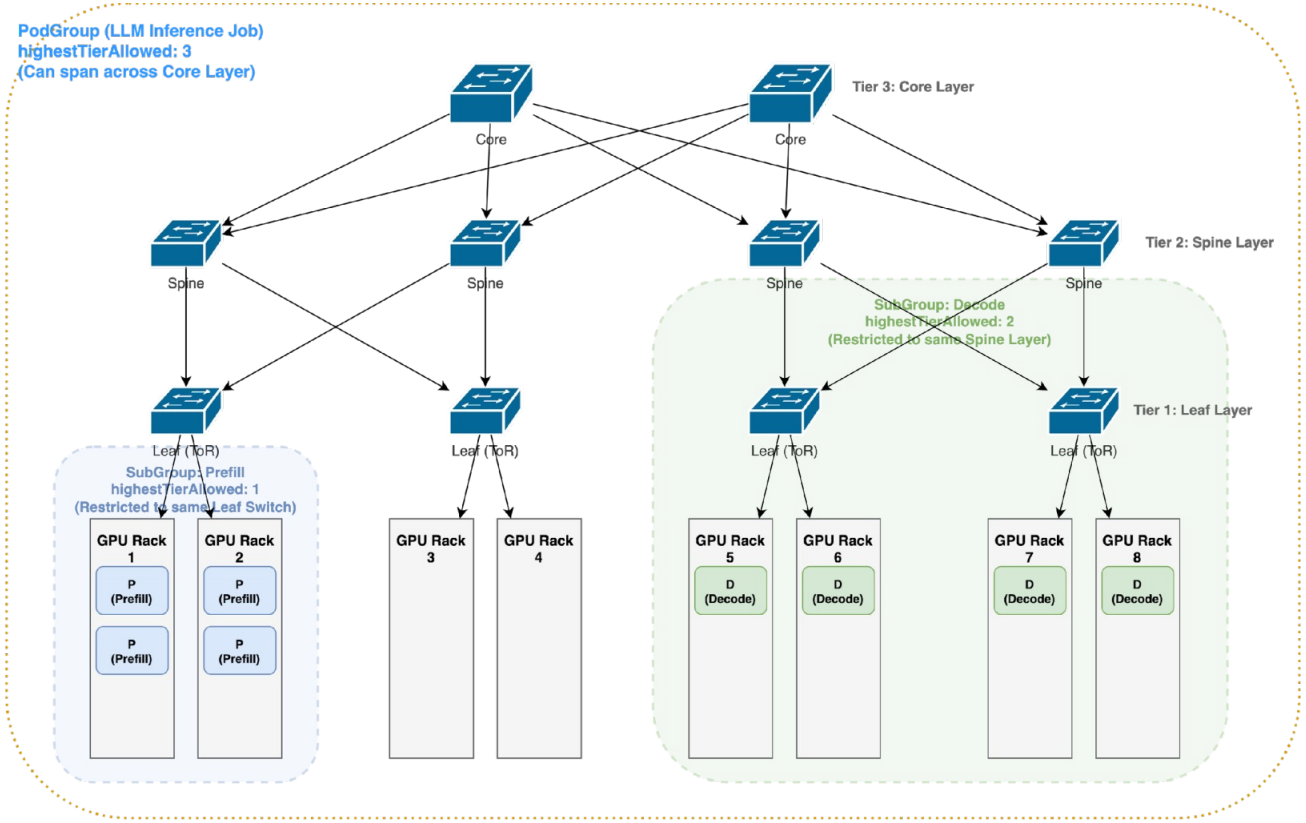
```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Leaf · HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

Parent: child HyperNodes · leaf: Kubernetes nodes

Place Groups at Different Topology Levels



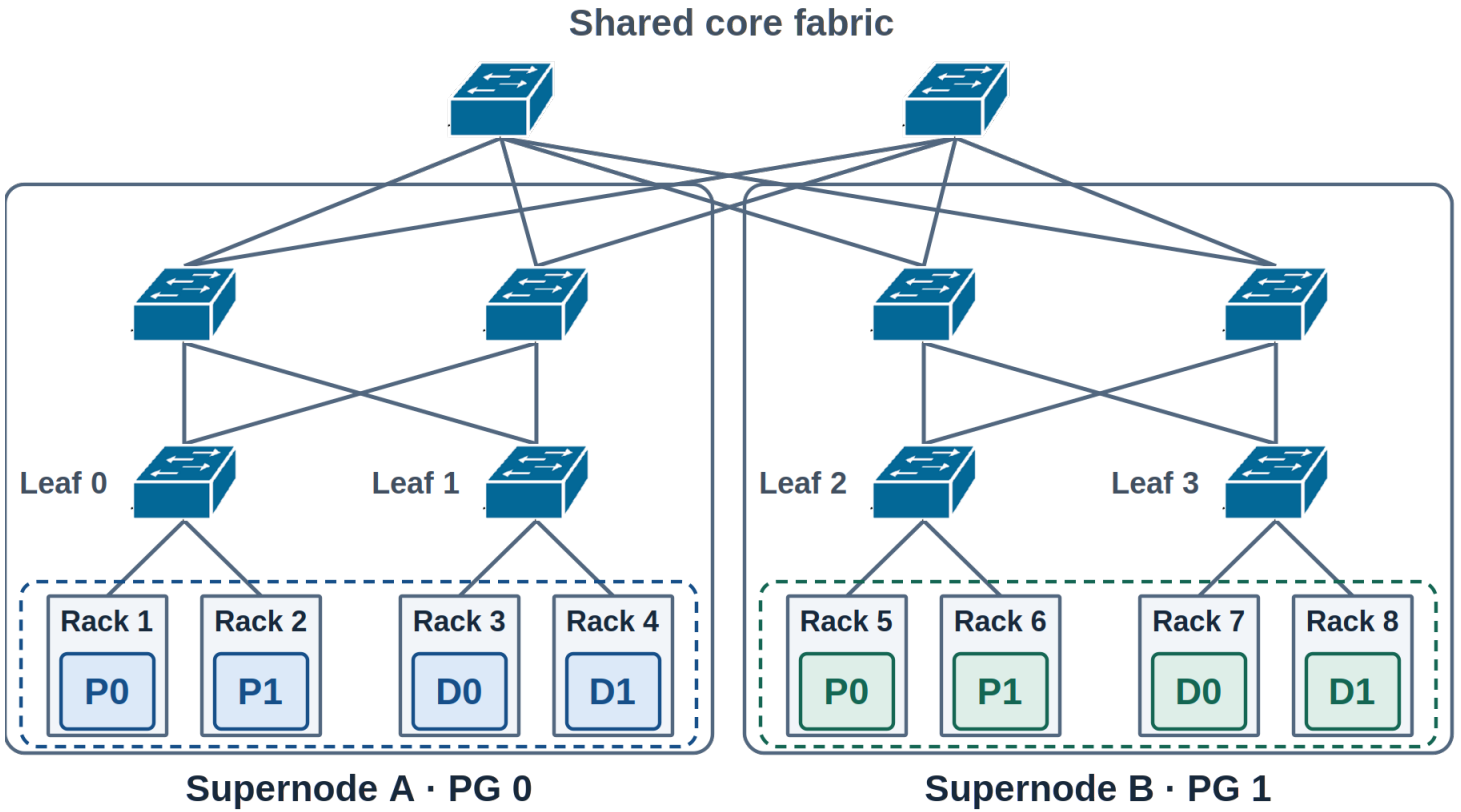
Whole-instance placement + role-specific locality

PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    highestTierName: spine
```

Keep Each Instance Together, Place Instances Apart

Same-label PodGroups → separate supernodes



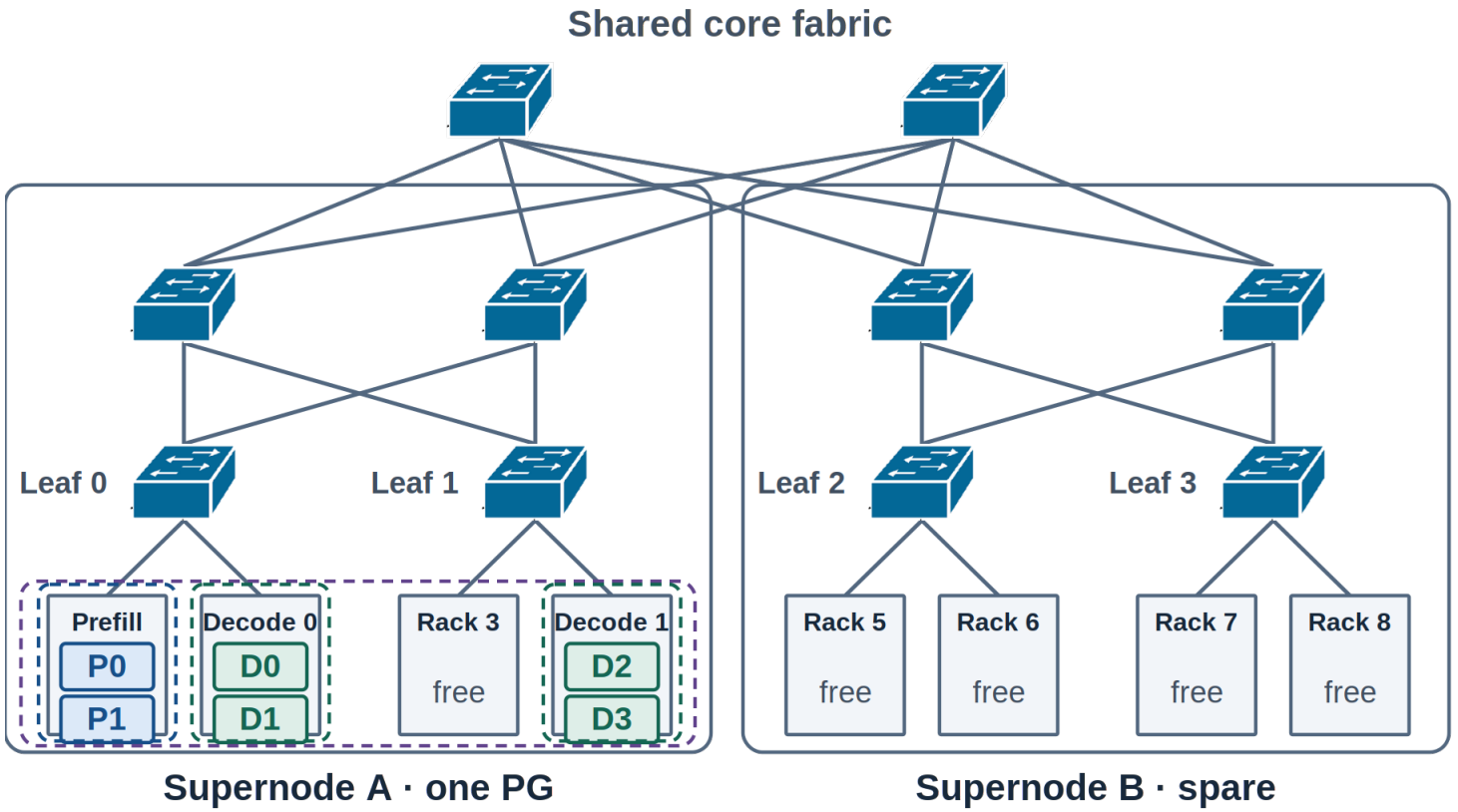
PodGroup.spec · key fields

```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

PG label: app=model-a

Instance-level HA across supernodes

Next: Relationships Between SubGroups



Affinity

Prefill + Decode: same supernode

Anti-affinity

Decode replicas: separate leaves

03

Ecosystem

Workload controllers carry scheduling intent.

01 Unified scheduling

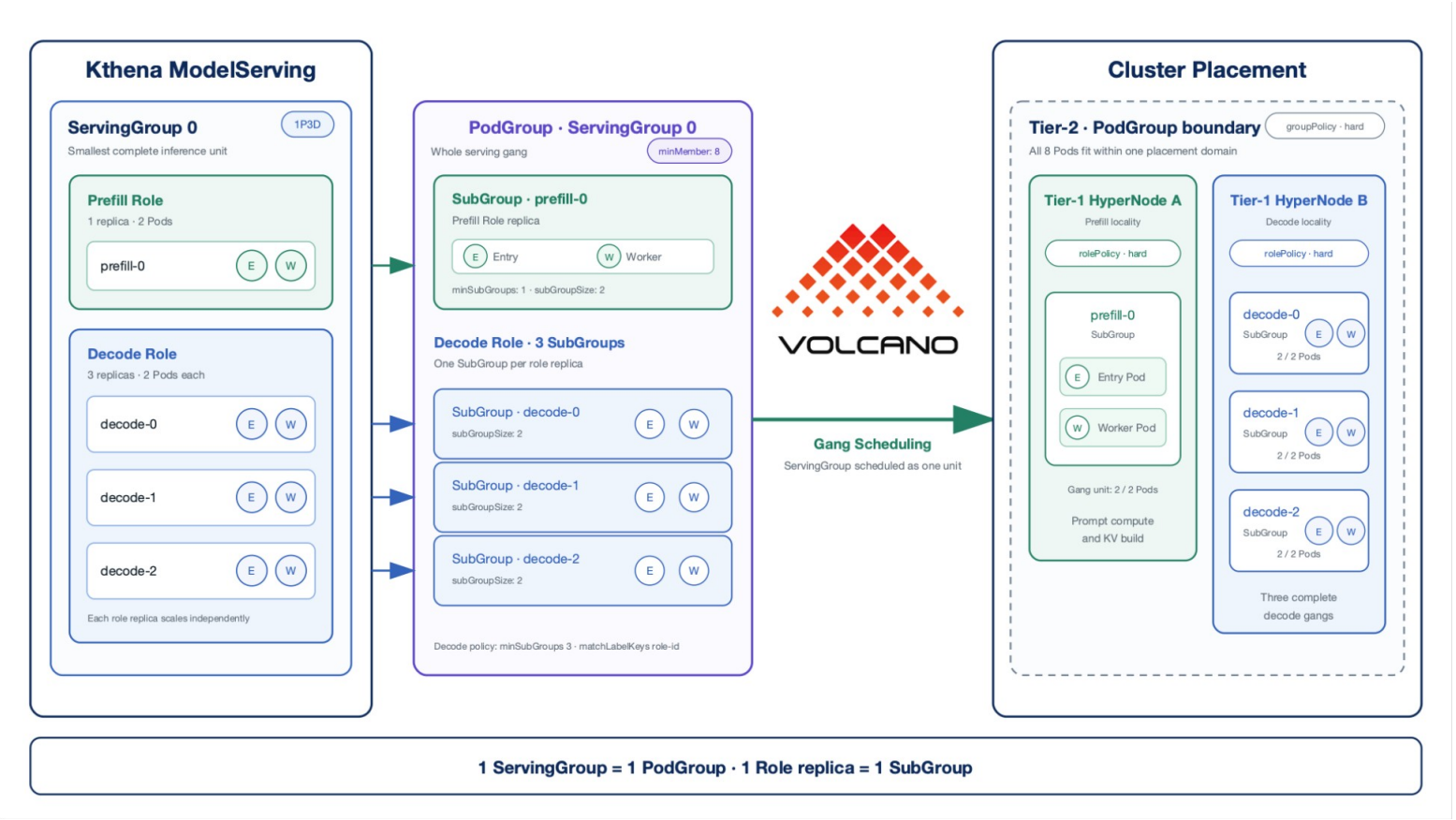
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Kthena: From Serving Roles to Scheduling Groups

ServingGroup → PodGroup · role replica → SubGroup



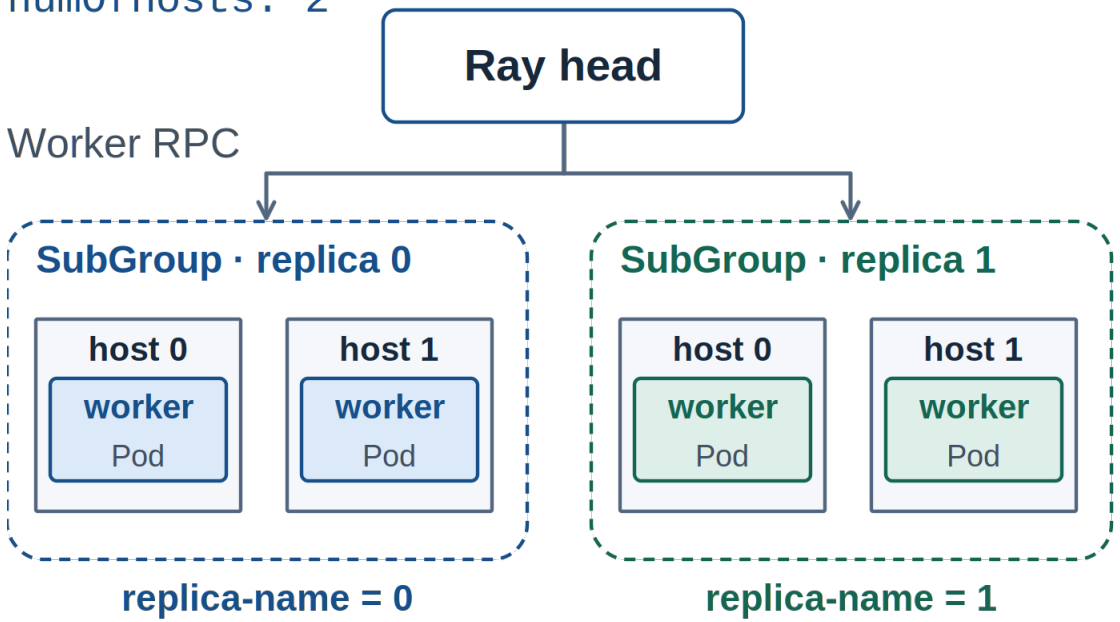
ModelServing.spec · excerpt

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

KubeRay: A Replica Can Span Multiple Pods

RayCluster · worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec · generated grouping

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

Replica label → one complete SubGroup

Minimum workers = 2 replicas × 2 hosts

Grove: Carry PodGang Structure into Volcano



Grove PodGang → **Volcano PodGroup**

PodGang.spec.podGroups

prefill `minReplicas: 2`

Pod 0 | Pod 1

decode `minReplicas: 4`

Pod 0 | Pod 1 | Pod 2 | Pod 3

SubGroup policy

SubGroup policy

PodGroup.spec.subGroupPolicy

prefill `subGroupSize: 2`

`minSubGroups: 1`

decode `subGroupSize: 4`

`minSubGroups: 1`

Minimum Pods per role

2 Prefill + 4 Decode Pods → `minMember: 6`

Next: RBG, Kubeflow, and LWS



RBG

In progress

RoleInstance → SubGroup

Replica identity + gang constraints



Kubeflow

Planned

Training groups + topology

Trainer jobs → scheduling groups



LWS

Planned

Multi-role / multi-group

Leader / worker roles + group relations

04

Benchmark framework & progress

Measure behavior across workload shapes.

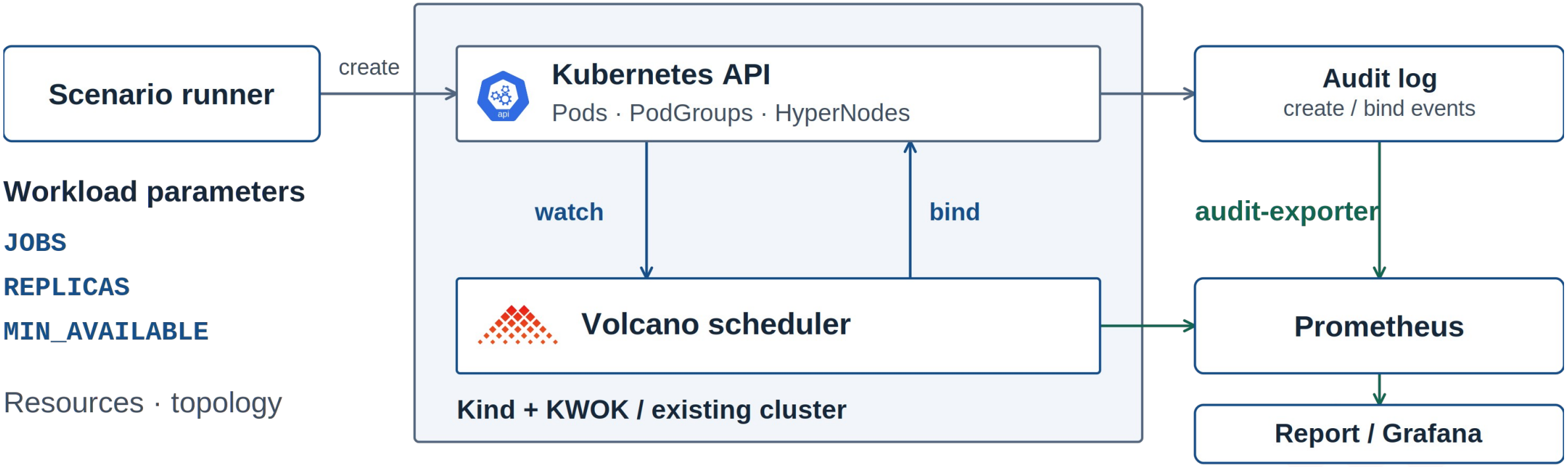
01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

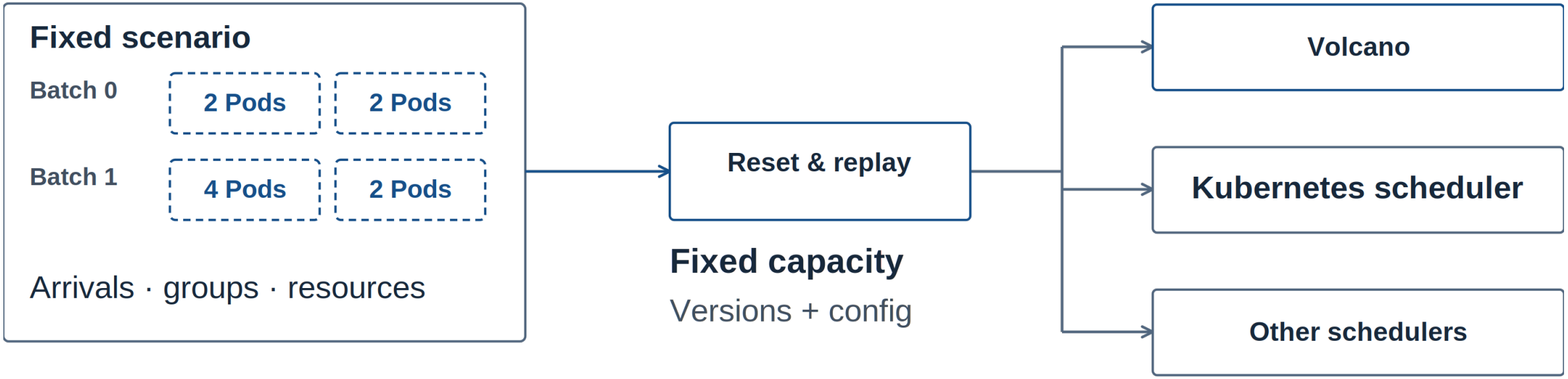
04 Benchmark framework & progress

A Repeatable Scheduling Benchmark Framework



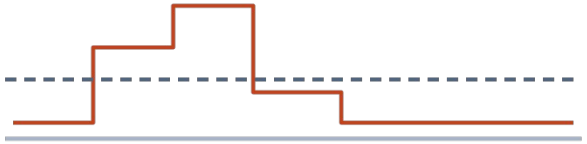
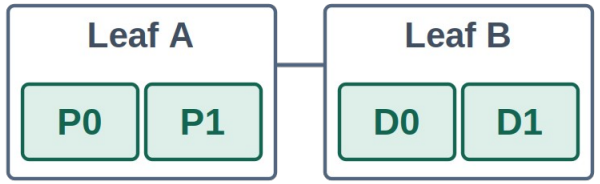
Scheduling latency · binding throughput

Next: Cross-Scheduler Comparisons



Latency · throughput · complete gangs · placement

Next: Workload and Topology Test Matrix

Scenario		Variables	Metrics
Bursts		**Arrival rate · capacity** Pool capacity requests	**Tail latency** Backlog recovery
Gang size		**Group size · fragmentation** Large / small gangs	**Complete gangs** Binding throughput
Topology		**Tier limits · domains** Gang · anti-affinity	**Valid placement** Gang latency

What We Will Work On Next

Workstream	Next	Validation
Group placement	**SubGroup relationships** Affinity / anti-affinity	Locality Replica isolation
Ecosystem	**RBG · Kubeflow · LWS** Multi-group integration	Group identity Placement intent
Benchmarks	**Cross-scheduler testing** Best-fit scenarios · topology	Latency · throughput Placement correctness

Volcano community



Thank you



volcano.sh



GitHub



Slack

Questions?